Claude Code Session Transcript

Project: cs415-hw0 — Mobile Weather Prototype
Repository: github.com/nkabra1/cs415-hw0 | Date: 2026-08-25

---

1. Initial Request

USER

Task:
Prototype a mobile webpage that shows local weather.

Requirements:
- should be a functional prototype that pulls live weather data
- should also be an interface prototype with plausibly permanent interface elements like icons, buttons, etc
- webpage should address 4 needs:
1. check the current temperature
2. see whether you need to bring a raincoat or umbrella if you are going out for the next few hours
3. give the current air quality index and the humidity level
4. give the current wind direction and speed

Design Style:
- should be black dark theme with colorful accents, icons should be colored but not emojis. the cards and panels should be sharp and not rounded.
[Request interrupted by user]
i want to host this on github pages so make this as easy and simple as possible there is no need to use react or flutter or any other framework.

■ *Claude entered Plan Mode to design an approach before writing any code (no edits or commands are executed while planning).*

---

2. Planning & Clarification

■ *The project directory contained only an empty git repository, so Claude skipped codebase exploration and asked three clarifying questions instead of guessing:*

CLAUDE

1. What temperature/wind units should the page use? — options: Fahrenheit / mph, Celsius / km/h, Auto-detect from browser locale.
2. If the visitor denies/lacks geolocation, how should the page get a location? — options: Manual city search box (Recommended), Fixed default city.
3. Single self-contained file, or separate HTML/CSS/JS files? — options: Single index.html (Recommended), Separate index.html/styles.css/app.js.

USER

Answered: "Fahrenheit / mph" for units; "use device location for the city, also allow for manual city search" for the fallback question; "Single index.html (Recommended)" for file layout.

■ *Claude wrote a plan (data source, page structure, visual design, files to create, verification steps) and requested approval via ExitPlanMode.*

USER

everything should fit on the page and be visible without scrolling

■ *The plan was rejected with that feedback. Claude revised it: instead of a single stacked column of 4 cards, the plan switched to a 2×2 CSS grid pinned to 100dvh with overflow:hidden, a 4-hour (not 6-hour) precipitation strip, and clamp()-based font sizing so the page fits without scrolling even on shorter phones. The revised plan was re-submitted and approved by the user.*

---

3. Approved Plan (verbatim)

plans/twinkly-soaring-quill.md

```
# Mobile Weather Prototype — Single-page GitHub Pages site

## Context

The repo (`cs415-hw0`) is currently empty (just `.git`). The goal is a mobile-first weather
webpage prototype that is:
- **Functional**: pulls live weather data from a real API.
- **Interface-level**: has plausibly-permanent chrome (icons, buttons, header) around the data,
  not just raw numbers.
- Deployable on **GitHub Pages** with zero build step, zero framework, and zero server-side
  secrets (GitHub Pages only serves static files, so any API key would be exposed client-side).

It must cover 4 needs: current temperature, rain/umbrella advice for the next few hours,
AQI + humidity, and wind direction/speed. Style: dark theme, colorful accents, sharp
(non-rounded) cards/panels, custom colored icons (no emoji).

User decisions from clarification:
- Units: **Fahrenheit / mph**.
- Location: use **device geolocation by default**, but always also offer a **manual city
  search box** (not just as a fallback — a visible, permanent UI element).
- Layout: **single self-contained `index.html`** (inline `<style>` and `<script>`, no external
  files) — simplest possible GitHub Pages setup.
- **No scrolling**: the entire page (header + all 4 cards) must fit within one mobile viewport
  with no vertical scroll.

## Approach

### Data source: Open-Meteo (no API key required)
Open-Meteo is free, keyless, and CORS-enabled — the only realistic choice for a pure static
GitHub Pages site (any keyed API like OpenWeatherMap would leak the key in client JS).

- **Geocoding (city search)**: `https://geocoding-api.open-meteo.com/v1/search?name={query}&count=5`
- **Reverse geocoding (label device-location coords)**: BigDataCloud's free reverse-geocode
  client endpoint `https://api.bigdatacloud.net/data/reverse-geocode-client?latitude=&longitude=&localityLanguage=en`
  (free, keyless, CORS-enabled — used only to display a human-readable place name; if it fails,
  fall back to showing lat/lon rounded to 2 decimals so the page still works).
- **Forecast**: `https://api.open-meteo.com/v1/forecast?latitude=&longitude=&current=temperature_2m,apparent_temperatu
re,relative_humidity_2m,precipitation,weather_code,wind_speed_10m,wind_direction_10m&hourly=precipitation_probability,
weather_code&forecast_days=1&temperature_unit=fahrenheit&wind_speed_unit=mph&timezone=auto`
- **Air quality**: `https://air-quality-api.open-meteo.com/v1/air-quality?latitude=&longitude=&current=us_aqi,pm2_5&ti
mezone=auto`

### Page structure (`index.html`, single file)

**No-scroll layout strategy**: the whole app is pinned to `100dvh` (with `100vh` fallback),
using a `display: flex; flex-direction: column` page with `overflow: hidden` on `html`/`body`.
The 4 cards are laid out as a **2×2 CSS grid** (`grid-template-columns: 1fr 1fr;
grid-template-rows: 1fr 1fr`) filling the remaining space below a compact header, rather than a
single stacked column — a 4-card vertical stack would overflow a typical phone screen once the
hourly-precipitation strip and header are included. Typography, padding, and the hourly-strip
bar count are kept compact (e.g. next-4-hours instead of 6) specifically to make each card's
content fit its grid cell without internal scrolling. Card search-results dropdown is
absolutely-positioned (overlaying content) so opening it never pushes layout height past the
viewport.

**Header** (compact, fixed height)
- App title/wordmark + a small colored "GPS/locate" icon button (re-requests geolocation) and
  a search icon button that toggles a text input (city search, using the geocoding endpoint,
  showing up to 5 matching results in an overlay list).
- Current resolved location name + last-updated time, single line, truncated with ellipsis if
  long.

**Card 1 — Current Temperature**
- Large temperature reading, "feels like", short condition text derived from Open-Meteo's
  `weather_code` (WMO code → label + icon mapping), colored sun/cloud/etc. SVG icon. Accent:
```

```
  amber/cyan gradient depending on condition.

**Card 2 — Rain / Umbrella advisory**
- Headline recommendation ("Bring an umbrella" / "No rain expected") based on max
  `precipitation_probability` over the next 4 hourly entries.
- Row of 4 compact hourly blocks (hour label, precip % as a small vertical bar, droplet icon)
  so the "next few hours" trend is visible, not just a single verdict, sized to fit the grid
  cell without overflow.
- Accent: blue.

**Card 3 — Air Quality & Humidity**
- US AQI value + category label (Good/Moderate/Unhealthy for Sensitive Groups/Unhealthy/Very
  Unhealthy/Hazardous) with the icon/accent colored per EPA-style breakpoints (green → maroon).
- Humidity percentage with a droplet/gauge icon.
- Accent: green→red scale driven by AQI value itself (data-driven accent, not fixed).

**Card 4 — Wind**
- Wind speed (mph) + compass-direction label (N/NE/E/.../NW) computed from the wind-direction
  degrees.
- A compass-rose SVG icon whose needle is rotated via CSS `transform: rotate()` to the live wind
  direction.
- Accent: purple/orange.

**States**: loading skeletons for each card while fetches are in flight; inline error text
(e.g. "Location permission denied — search for a city instead") without breaking layout; the
page must degrade gracefully to the search box if geolocation is denied/unavailable.

### Visual design
- Root background near-black (`#0b0b0d`), card background slightly lighter charcoal
  (`#17171a`), 1px hairline borders, `border-radius: 0` everywhere (sharp corners) per the style
  requirement.
- Accent colors per card as above, applied to icon strokes/fills, the card's top border or
  accent bar, and key numbers — not as full-card background fills (keeps it dark-theme).
- All icons are hand-written inline `<svg>` (thermometer, umbrella/cloud-rain, gauge, wind
  compass, search, locate-me, droplet) styled via `currentColor`/CSS — no emoji, no icon-font
  CDN dependency, keeping the single file self-contained and offline-buildable.
- Mobile-first: content column `max-width: 480px`, centered, comfortable tap targets (≥44px) for
  the header buttons and search results list, responsive down to narrow phone widths.
- System font stack (no webfont CDN) to keep the page dependency-free.
- Sizing uses `clamp()`/`min()` for font sizes and `fr` units for the grid so the fixed-height,
  no-scroll layout still degrades gracefully across common phone heights (~650px–930px) instead
  of clipping content on smaller devices.

### File to create
- [index.html](index.html) — the entire prototype (structure + `<style>` + `<script>`).
- [README.md](README.md) — brief instructions: what the page does, and the 2-step GitHub Pages
  setup (push to `main`, enable Pages → Deploy from branch → `main` / root).

No other files, no `package.json`, no build tooling — plain HTML/CSS/JS only, per the request.

## Verification
- Open `index.html` directly via the Browser tool (or a `preview_start` static server) and:
  - Confirm/deny geolocation and check both paths render correctly.
  - Use the manual search box to look up a city and confirm the cards update.
  - Confirm all 4 cards populate with live data (temp, rain advisory + hourly strip, AQI +
    humidity, wind speed/direction with rotated compass icon).
  - Resize to a mobile viewport (375×812, plus a shorter one like 360×650) and confirm layout,
    sharp corners, colored (non-emoji) icons, and **zero vertical scroll** (all 4 cards + header
    fully visible at once) in the dark theme.
  - Check the browser console for fetch/CORS errors.
```

---

4. Implementation & Testing

■ *Claude created index.html (single self-contained file: markup + inline <style> + inline <script>) and README.md, then tested the page in an embedded browser before reporting completion.*

■ *Testing performed: served the folder with `python -m http.server` via a temporary .claude/launch.json config; loaded the page at a 375×812 mobile viewport; confirmed the geolocation-denied path shows an inline error and auto-opens the city search panel; searched "Chicago" and selected the top result, confirming all four cards populated with live Open-Meteo data (65°F Overcast / feels like 66°F; "No rain expected" with 0% bars for the next 4 hours; AQI 56 “Moderate” in amber; 79% humidity; 7 mph wind from S (172°) with the compass needle rotated to match); re-tested at a shorter 360×650 viewport to confirm the no-scroll layout still holds; checked the browser console for errors (none found).*

CLAUDE

Built and verified. Summary:

Files created (repo root):

- index.html (index.html) — the entire prototype: markup, styles, and JS in one file
- README.md (README.md) — GitHub Pages deploy instructions (2 steps: push to main, enable Pages)
- .claude/launch.json — lets you re-preview locally via the Browser tool later (not needed for GitHub Pages, doesn't affect deployment)

How it works
- Live data from Open-Meteo (https://open-meteo.com/) (forecast, air quality, city-search geocoding) — free, no API key, since GitHub Pages can't hide secrets client-side. Device-location reverse geocoding uses BigDataCloud's free keyless endpoint.
- Requests device geolocation on load; if denied/unavailable it shows an inline error and opens the always-available manual city search (debounced, hits Open-Meteo's geocoding API, click a result to load its weather).
- Four cards in a 2×2 grid, pinned to the viewport height (100dvh, overflow:hidden) so nothing scrolls: Temperature (+ condition icon), Umbrella? (verdict + next-4-hours precip-% bars), Air & Humidity (US AQI, colored by EPA severity band, + humidity), Wind (speed, compass-direction label, needle rotated to live wind degrees).
- Dark theme (#0b0b0d/#141416), sharp corners (border-radius:0 throughout), hand-drawn colored SVG line icons (no emoji), per-card accent colors (cyan/blue/AQI-driven/purple).

Verified in-browser at 375×812 and a shorter 360×650: live search for "Chicago" correctly rendered 65°F Overcast, "No rain expected" with 0% hourly bars, AQI 56 Moderate (amber), 79% humidity, 7 mph wind from S with the compass needle rotated — all four cards visible with zero scrolling, no console errors.

To publish, just push this repo to GitHub and turn on Pages (Settings → Pages → Deploy from branch → main / root) — full steps are in the README.

---

5. Git — Commit

USER

Commit the working tree changes with a sensible message.

■ *Claude ran `git status`, `git diff --stat`, and `git log` first. All three files were already staged for this repo's initial commit. Commit created:*

git commit

```
commit 312ca3d (main, root-commit)
Add mobile weather prototype for GitHub Pages

Single-file HTML/CSS/JS page (no framework, no build step) that pulls
live data from Open-Meteo to show current temperature, a rain/umbrella
advisory for the next few hours, AQI + humidity, and wind speed/direction,
all in a no-scroll dark-themed 2x2 card layout.

 3 files changed, 791 insertions(+)
 create mode 100644 .claude/launch.json
 create mode 100644 README.md
 create mode 100644 index.html
```

CLAUDE

Committed as 312ca3d — the initial commit with index.html, README.md, and .claude/launch.json. Working tree is clean.

---

6. Git — Push

USER

push it to GitHub

■ *Claude checked `git remote -v`, found origin already configured (git@github.com-school:nkabra1/cs415-hw0.git), and ran `git push -u origin main`.*

CLAUDE

Pushed. main is now up on nkabra1/cs415-hw0 (https://github.com/nkabra1/cs415-hw0) with upstream tracking set.

To finish deploying: go to the repo's Settings → Pages, set Source to "Deploy from a branch", pick main / root, and save — it'll be live at https://nkabra1.github.io/cs415-hw0/ shortly after.

---

7. Transcript Export Request

USER

/export cs415-hw0-claude-transcript.md

■ *System response: "/export isn't available in this environment."*

USER

i want the whole transcript for this session as a pdf so i can submit it with my assignment

■ *Claude reconstructed this transcript from the session and rendered it to PDF (this document) using Python + reportlab, then delivered it as a file.*

Appendix A: index.html (full source)

The complete, final contents of the single-file prototype committed and pushed to the repository.

index.html

```
<!DOCTYPE html>
<html lang="en">
<head>
<meta charset="UTF-8" />
<meta name="viewport" content="width=device-width, initial-scale=1, maximum-scale=1, user-scalable=no, viewport-fit=co
ver" />
<meta name="theme-color" content="#0b0b0d" />
<title>Weather</title>
<style>
  :root {
    --bg: #0b0b0d;
    --card-bg: #141416;
    --card-bg-2: #1a1a1d;
    --border: #29292d;
    --text: #f4f4f5;
    --text-dim: #8c8c94;
    --text-faint: #5c5c63;
    --accent-temp: #22d3ee;
    --accent-rain: #3b82f6;
    --accent-wind: #c084fc;
    --accent-aqi: #22c55e;
  }

  * { box-sizing: border-box; }

  html, body {
    height: 100%;
    margin: 0;
    overflow: hidden;
    background: var(--bg);
    color: var(--text);
    font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", Roboto, Helvetica, Arial, sans-serif;
  }

  #app {
    height: 100vh;
    height: 100dvh;
    max-width: 480px;
    margin: 0 auto;
    display: flex;
    flex-direction: column;
    padding: 10px 10px calc(10px + env(safe-area-inset-bottom, 0px));
    gap: 8px;
    position: relative;
  }

  /* ---------- header ---------- */
  header {
    flex: 0 0 auto;
    border: 1px solid var(--border);
    background: var(--card-bg);
    padding: 10px 12px;
    position: relative;
  }

  .title-row {
    display: flex;
    align-items: center;
    justify-content: space-between;
  }

  .title-row h1 {
    font-size: 15px;
    font-weight: 700;
    letter-spacing: 0.06em;
    text-transform: uppercase;
    margin: 0;
    color: var(--text);
  }

  .header-actions { display: flex; gap: 6px; }

  .icon-btn {
    width: 30px;
    height: 30px;
    display: flex;
    align-items: center;
    justify-content: center;
    background: transparent;
    border: 1px solid var(--border);
    color: var(--text-dim);
```

```
  cursor: pointer;
  padding: 0;
}
.icon-btn:hover { color: var(--text); border-color: #3d3d42; }
.icon-btn.active { color: var(--accent-temp); border-color: var(--accent-temp); }
.icon-btn svg { width: 16px; height: 16px; }

.location-row {
  margin-top: 6px;
  display: flex;
  align-items: baseline;
  justify-content: space-between;
  gap: 8px;
  font-size: 12px;
}
#locationName {
  color: var(--text);
  font-weight: 600;
  white-space: nowrap;
  overflow: hidden;
  text-overflow: ellipsis;
}
#updatedAt { color: var(--text-faint); flex-shrink: 0; font-size: 10.5px; }

.search-panel {
  position: absolute;
  top: calc(100% + 4px);
  left: 0;
  right: 0;
  background: var(--card-bg-2);
  border: 1px solid var(--border);
  padding: 8px;
  z-index: 20;
}
.search-panel.hidden { display: none; }
#searchInput {
  width: 100%;
  background: var(--bg);
  border: 1px solid var(--border);
  color: var(--text);
  padding: 8px 9px;
  font-size: 13px;
  outline: none;
}
#searchInput:focus { border-color: var(--accent-temp); }
#searchResults { list-style: none; margin: 6px 0 0; padding: 0; max-height: 40vh; overflow-y: auto; }
#searchResults li {
  padding: 8px 9px;
  font-size: 12.5px;
  color: var(--text-dim);
  border-top: 1px solid var(--border);
  cursor: pointer;
}
#searchResults li:first-child { border-top: none; }
#searchResults li:hover { background: var(--bg); color: var(--text); }
#searchStatus { font-size: 11.5px; color: var(--text-faint); padding: 6px 2px 0; }

/* ---------- grid ---------- */
main.grid {
  flex: 1 1 auto;
  min-height: 0;
  display: grid;
  grid-template-columns: 1fr 1fr;
  grid-template-rows: 1fr 1fr;
  gap: 8px;
}

.card {
  background: var(--card-bg);
  border: 1px solid var(--border);
  border-top: 3px solid var(--accent);
  padding: 10px 11px;
  display: flex;
  flex-direction: column;
  min-height: 0;
  min-width: 0;
  overflow: hidden;
  position: relative;
}

.card-head {
  display: flex;
  align-items: center;
  gap: 6px;
  flex: 0 0 auto;
  color: var(--accent);
}
.card-head svg { width: 15px; height: 15px; flex-shrink: 0; }
.card-head h2 {
  margin: 0;
```

```
  font-size: 10.5px;
  font-weight: 700;
  letter-spacing: 0.07em;
  text-transform: uppercase;
  color: var(--text-dim);
}

.card-body {
  flex: 1 1 auto;
  min-height: 0;
  display: flex;
  flex-direction: column;
  justify-content: center;
  gap: 2px;
}

/* --- temp card --- */
.temp-main {
  display: flex;
  align-items: flex-start;
  line-height: 1;
}
#tempValue { font-size: clamp(30px, 11vw, 46px); font-weight: 700; color: var(--text); }
.temp-main .unit { font-size: clamp(14px, 4vw, 20px); font-weight: 600; color: var(--text-dim); margin-top: 3px; }
#tempCondition { font-size: 12.5px; color: var(--text); margin-top: 2px; }
.temp-feels { font-size: 11px; color: var(--text-faint); margin-top: 1px; }
#conditionIcon {
  position: absolute;
  top: 8px;
  right: 9px;
  width: 22px;
  height: 22px;
  color: var(--accent-temp);
  opacity: 0.9;
}
#conditionIcon svg { width: 100%; height: 100%; }

/* --- rain card --- */
#rainVerdict {
  font-size: clamp(12.5px, 3.6vw, 14.5px);
  font-weight: 700;
  color: var(--text);
  margin-bottom: 6px;
}
#rainVerdict.wet { color: var(--accent-rain); }
.hourly-strip {
  display: flex;
  justify-content: space-between;
  gap: 5px;
  flex: 1 1 auto;
  min-height: 0;
  align-items: flex-end;
}
.hour-block {
  flex: 1 1 0;
  display: flex;
  flex-direction: column;
  align-items: center;
  justify-content: flex-end;
  gap: 3px;
  min-width: 0;
}
.hour-block .hb-pct { font-size: 10px; color: var(--text-dim); }
.hour-block .hb-bar-track {
  width: 100%;
  max-width: 18px;
  height: 34px;
  background: var(--card-bg-2);
  border: 1px solid var(--border);
  display: flex;
  align-items: flex-end;
}
.hour-block .hb-bar-fill {
  width: 100%;
  background: var(--accent-rain);
}
.hour-block .hb-time { font-size: 9.5px; color: var(--text-faint); }

/* --- aqi card --- */
.aqi-row { display: flex; align-items: baseline; gap: 7px; }
#aqiValue { font-size: clamp(26px, 9vw, 38px); font-weight: 700; color: var(--accent-aqi); line-height: 1; }
#aqiLabel {
  font-size: 10.5px;
  font-weight: 700;
  text-transform: uppercase;
  letter-spacing: 0.04em;
  color: var(--accent-aqi);
  border: 1px solid var(--accent-aqi);
  padding: 2px 5px;
}
```

```
  .aqi-caption { font-size: 10px; color: var(--text-faint); margin-top: 2px; }
  .humidity-row {
    display: flex;
    align-items: center;
    gap: 5px;
    font-size: 12px;
    color: var(--text-dim);
    margin-top: 8px;
    padding-top: 7px;
    border-top: 1px solid var(--border);
  }
  .humidity-row svg { width: 13px; height: 13px; color: #38bdf8; flex-shrink: 0; }
  .humidity-row #humidityValue { color: var(--text); font-weight: 700; }

  /* --- wind card --- */
  .wind-body { flex-direction: row; align-items: center; gap: 10px; }
  .compass { width: clamp(46px, 15vw, 64px); height: clamp(46px, 15vw, 64px); flex-shrink: 0; }
  .compass svg { width: 100%; height: 100%; }
  #compassNeedle { transform-origin: 50% 50%; transition: transform 0.4s ease; }
  .wind-info { display: flex; flex-direction: column; min-width: 0; }
  #windSpeed { font-size: clamp(22px, 7.5vw, 30px); font-weight: 700; color: var(--text); line-height: 1; }
  .wind-info .wind-unit { font-size: 12px; color: var(--text-dim); font-weight: 600; }
  #windDir { font-size: 12px; color: var(--accent-wind); font-weight: 700; margin-top: 3px; }

  .hidden { display: none !important; }

  #errorBanner {
    position: absolute;
    left: 10px;
    right: 10px;
    bottom: 10px;
    background: #1a1112;
    border: 1px solid #7f1d1d;
    color: #fca5a5;
    font-size: 11.5px;
    padding: 8px 10px;
    z-index: 30;
  }
</style>
</head>
<body>
<div id="app">

  <header>
    <div class="title-row">
      <h1>Weather</h1>
      <div class="header-actions">
        <button id="locateBtn" class="icon-btn" aria-label="Use my location" title="Use my location">
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"><circle c
x="12" cy="12" r="3"></circle><path d="M12 2v3M12 19v3M2 12h3M19 12h3"></path><circle cx="12" cy="12" r="8"></circle><
/svg>
        </button>
        <button id="searchBtn" class="icon-btn" aria-label="Search city" title="Search city">
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"><circle c
x="11" cy="11" r="7"></circle><path d="M21 21l-4.3-4.3"></path></svg>
        </button>
      </div>
    </div>
    <div class="location-row">
      <span id="locationName">Locating…</span>
      <span id="updatedAt"></span>
    </div>
    <div id="searchPanel" class="search-panel hidden">
      <input id="searchInput" type="text" placeholder="Search city…" autocomplete="off" />
      <div id="searchStatus"></div>
      <ul id="searchResults"></ul>
    </div>
  </header>

  <main class="grid">

    <section class="card" id="cardTemp" style="--accent:var(--accent-temp)">
      <div class="card-head">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-line
join="round"><path d="M14 14.76V3.5a2.5 2.5 0 0 0-5 0v11.26a4.5 4.5 0 1 0 5 0z"></path></svg>
        <h2>Temperature</h2>
      </div>
      <div class="card-body">
        <div class="temp-main"><span id="tempValue">--</span><span class="unit">°F</span></div>
        <div id="tempCondition">—</div>
        <div class="temp-feels">Feels like <span id="tempFeels">--°</span></div>
      </div>
      <div id="conditionIcon"></div>
    </section>

    <section class="card" id="cardRain" style="--accent:var(--accent-rain)">
      <div class="card-head">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-line
join="round"><path d="M12 3c-3 3-6 6.5-6 9.5A6 6 0 0 0 18 12.5C18 9.5 15 6 12 3z"></path><path d="M12 3v3"></path><pat
h d="M9 16.5v2M12 17.5v3M15 16.5v2"></path></svg>
```

```
        <h2>Umbrella?</h2>
      </div>
      <div class="card-body">
        <div id="rainVerdict">—</div>
        <div class="hourly-strip" id="hourlyStrip"></div>
      </div>
    </section>

    <section class="card" id="cardAQI" style="--accent:var(--accent-aqi)">
      <div class="card-head">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-line
join="round"><path d="M4 19a8 8 0 0 1 1 16 0"></path><path d="M12 19l4.5-7"></path><circle cx="12" cy="19" r="1"></circl
e></svg>
        <h2>Air &amp;amp; Humidity</h2>
      </div>
      <div class="card-body">
        <div class="aqi-row">
          <span id="aqiValue">--</span>
          <span id="aqiLabel">—</span>
        </div>
        <div class="aqi-caption">US AQI</div>
        <div class="humidity-row">
          <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-li
nejoin="round"><path d="M12 3s6 6.5 6 11a6 6 0 0 1-12 0c0-4.5 6-11 6-11z"></path></svg>
          <span id="humidityValue">--%</span> humidity
        </div>
      </div>
    </section>

    <section class="card" id="cardWind" style="--accent:var(--accent-wind)">
      <div class="card-head">
        <svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-line
join="round"><path d="M2 8h13a3 3 0 1 0-3-3"></path><path d="M2 13h17a3 3 0 1 1-3 3"></path><path d="M2 18h9a2.5 2.5 0
 1 1-2.5 2.5"></path></svg>
        <h2>Wind</h2>
      </div>
      <div class="card-body wind-body">
        <div class="compass">
          <svg viewBox="0 0 100 100" fill="none">
            <circle cx="50" cy="50" r="46" stroke="var(--border)" stroke-width="2"></circle>
            <circle cx="50" cy="50" r="46" stroke="currentColor" stroke-width="2" stroke-dasharray="2 6" opacity="0.5"
></circle>
            <text x="50" y="14" fill="currentColor" font-size="10" font-weight="700" text-anchor="middle">N</text>
            <text x="90" y="54" fill="var(--text-faint)" font-size="9" text-anchor="middle">E</text>
            <text x="50" y="94" fill="var(--text-faint)" font-size="9" text-anchor="middle">S</text>
            <text x="10" y="54" fill="var(--text-faint)" font-size="9" text-anchor="middle">W</text>
            <g id="compassNeedle">
              <polygon points="50,14 57,50 50,42 43,50" fill="currentColor"></polygon>
              <polygon points="50,86 57,50 50,58 43,50" fill="var(--text-faint)"></polygon>
            </g>
            <circle cx="50" cy="50" r="4" fill="var(--card-bg)" stroke="currentColor" stroke-width="2"></circle>
          </svg>
        </div>
        <div class="wind-info">
          <div><span id="windSpeed">--</span><span class="wind-unit"> mph</span></div>
          <div id="windDir">--</div>
        </div>
      </div>
    </section>

  </main>

  <div id="errorBanner" class="hidden"></div>
</div>

<script>
(function () {
  "use strict";

  var els = {
    locateBtn: document.getElementById("locateBtn"),
    searchBtn: document.getElementById("searchBtn"),
    searchPanel: document.getElementById("searchPanel"),
    searchInput: document.getElementById("searchInput"),
    searchResults: document.getElementById("searchResults"),
    searchStatus: document.getElementById("searchStatus"),
    locationName: document.getElementById("locationName"),
    updatedAt: document.getElementById("updatedAt"),
    errorBanner: document.getElementById("errorBanner"),
    tempValue: document.getElementById("tempValue"),
    tempCondition: document.getElementById("tempCondition"),
    tempFeels: document.getElementById("tempFeels"),
    conditionIcon: document.getElementById("conditionIcon"),
    rainVerdict: document.getElementById("rainVerdict"),
    hourlyStrip: document.getElementById("hourlyStrip"),
    aqiValue: document.getElementById("aqiValue"),
    aqiLabel: document.getElementById("aqiLabel"),
    humidityValue: document.getElementById("humidityValue"),
    windSpeed: document.getElementById("windSpeed"),
    windDir: document.getElementById("windDir"),
```

```
    compassNeedle: document.getElementById("compassNeedle"),
    cardAQI: document.getElementById("cardAQI")
  };

  var WMO = {
    0: { label: "Clear sky", icon: "sun" },
    1: { label: "Mostly clear", icon: "cloudsun" },
    2: { label: "Partly cloudy", icon: "cloudsun" },
    3: { label: "Overcast", icon: "cloud" },
    45: { label: "Fog", icon: "fog" },
    48: { label: "Fog", icon: "fog" },
    51: { label: "Light drizzle", icon: "rain" },
    53: { label: "Drizzle", icon: "rain" },
    55: { label: "Dense drizzle", icon: "rain" },
    56: { label: "Freezing drizzle", icon: "rain" },
    57: { label: "Freezing drizzle", icon: "rain" },
    61: { label: "Light rain", icon: "rain" },
    63: { label: "Rain", icon: "rain" },
    65: { label: "Heavy rain", icon: "rain" },
    66: { label: "Freezing rain", icon: "rain" },
    67: { label: "Freezing rain", icon: "rain" },
    71: { label: "Light snow", icon: "snow" },
    73: { label: "Snow", icon: "snow" },
    75: { label: "Heavy snow", icon: "snow" },
    77: { label: "Snow grains", icon: "snow" },
    80: { label: "Rain showers", icon: "rain" },
    81: { label: "Rain showers", icon: "rain" },
    82: { label: "Violent showers", icon: "rain" },
    85: { label: "Snow showers", icon: "snow" },
    86: { label: "Snow showers", icon: "snow" },
    95: { label: "Thunderstorm", icon: "storm" },
    96: { label: "Thunderstorm", icon: "storm" },
    99: { label: "Thunderstorm", icon: "storm" }
  };

  var CONDITION_ICONS = {
    sun: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"><circle c
x="12" cy="12" r="4.5"></circle><path d="M12 2v2.5M12 19.5V22M4.2 4.2l1.8 1.8M18 18l1.8 1.8M2 12h2.5M19.5 12H22M4.2 19
.8L6 18M18 6l1.8-1.8"></path></svg>',
    cloudsun: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stro
ke-linejoin="round"><circle cx="8" cy="8" r="3"></circle><path d="M8 2.5v1.5M8 12v1.5M2.5 8H4M12.5 8H14M4 4l1 1M12 4l-
1 1"></path><path d="M8.5 12.5h7a3.5 3.5 0 1 0-1.2-6.8A5 5 0 0 0 5 9.3 3 3 0 0 0 6 15h9.5"></path></svg>',
    cloud: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"><path d="M6.5 18.5h11a4 4 0 0 0 .6-7.95 6 6 0 0 0-11.6 1.9A3.75 3.75 0 0 0 6.5 18.5z"></path></svg>',
    fog: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round"><path d="
M4 8h11a3.5 3.5 0 1 0-1.05-6.85A5 5 0 0 0 4.5 5.8"></path><path d="M3 12h18M3 16h18M3 20h13"></path></svg>',
    rain: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-l
inejoin="round"><path d="M6.5 13.5h11a4 4 0 0 0 .6-7.95 6 6 0 0 0-11.6 1.9A3.75 3.75 0 0 0 6.5 13.5z"></path><path d="
M8 17.5l-1 2.5M12 17.5l-1 2.5M16 17.5l-1 2.5"></path></svg>',
    snow: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-l
inejoin="round"><path d="M6.5 13h11a4 4 0 0 0 .6-7.95 6 6 0 0 0-11.6 1.9A3.75 3.75 0 0 0 6.5 13z"></path><path d="M12
16v6M9.2 17.5l5.6 3M14.8 17.5l-5.6 3"></path></svg>',
    storm: '<svg viewBox="0 0 24 24" fill="none" stroke="currentColor" stroke-width="2" stroke-linecap="round" stroke-
linejoin="round"><path d="M6.5 12.5h11a4 4 0 0 0 .6-7.95 6 6 0 0 0-11.6 1.9A3.75 3.75 0 0 0 6.5 12.5z"></path><path d=
"M13 13l-3 5h3l-2 4"></path></svg>'
  };

  var COMPASS_DIRS = ["N","NNE","NE","ENE","E","ESE","SE","SSE","S","SSW","SW","WSW","W","WNW","NW","NNW"];

  var AQI_LEVELS = [
    { max: 50, label: "Good", color: "#22c55e" },
    { max: 100, label: "Moderate", color: "#eab308" },
    { max: 150, label: "Sensitive", color: "#f97316" },
    { max: 200, label: "Unhealthy", color: "#ef4444" },
    { max: 300, label: "Very Unhealthy", color: "#a855f7" },
    { max: Infinity, label: "Hazardous", color: "#f472b6" }
  ];

  function showError(msg) {
    els.errorBanner.textContent = msg;
    els.errorBanner.classList.remove("hidden");
  }
  function clearError() {
    els.errorBanner.classList.add("hidden");
  }

  function windDirLabel(deg) {
    var idx = Math.round(deg / 22.5) % 16;
    return COMPASS_DIRS[idx];
  }

  function aqiLevel(val) {
    for (var i = 0; i < AQI_LEVELS.length; i++) {
      if (val <= AQI_LEVELS[i].max) return AQI_LEVELS[i];
    }
    return AQI_LEVELS[AQI_LEVELS.length - 1];
  }

  function formatHour(iso) {
    var d = new Date(iso);
```

```
  var h = d.getHours();
  var suffix = h >= 12 ? "PM" : "AM";
  var h12 = h % 12;
  if (h12 === 0) h12 = 12;
  return h12 + suffix;
}

// ---------------- rendering ----------------

function renderTemp(current) {
  var wmo = WMO[current.weather_code] || { label: "—", icon: "cloud" };
  els.tempValue.textContent = Math.round(current.temperature_2m);
  els.tempCondition.textContent = wmo.label;
  els.tempFeels.textContent = Math.round(current.apparent_temperature) + "°F";
  els.conditionIcon.innerHTML = CONDITION_ICONS[wmo.icon] || CONDITION_ICONS.cloud;
}

function renderRain(hourly, nowIso) {
  var times = hourly.time;
  var probs = hourly.precipitation_probability;
  var startIdx = 0;
  for (var i = 0; i < times.length; i++) {
    if (times[i] >= nowIso) { startIdx = i; break; }
  }
  var slice = [];
  for (var j = startIdx; j < Math.min(startIdx + 4, times.length); j++) {
    slice.push({ time: times[j], pct: probs[j] });
  }
  var maxPct = slice.reduce(function (m, s) { return Math.max(m, s.pct); }, 0);

  if (maxPct >= 40) {
    els.rainVerdict.textContent = "Bring an umbrella";
    els.rainVerdict.classList.add("wet");
  } else if (maxPct >= 15) {
    els.rainVerdict.textContent = "Slight chance of rain";
    els.rainVerdict.classList.remove("wet");
  } else {
    els.rainVerdict.textContent = "No rain expected";
    els.rainVerdict.classList.remove("wet");
  }

  els.hourlyStrip.innerHTML = "";
  slice.forEach(function (s) {
    var block = document.createElement("div");
    block.className = "hour-block";
    var barHeight = Math.max(4, Math.round((s.pct / 100) * 34));
    block.innerHTML =
      '<span class="hb-pct">' + s.pct + '%</span>' +
      '<span class="hb-bar-track"><span class="hb-bar-fill" style="height:' + barHeight + 'px"></span></span>' +
      '<span class="hb-time">' + formatHour(s.time) + '</span>';
    els.hourlyStrip.appendChild(block);
  });
}

function renderAQI(usAqi, humidity) {
  var level = aqiLevel(usAqi);
  els.aqiValue.textContent = Math.round(usAqi);
  els.aqiValue.style.color = level.color;
  els.aqiLabel.textContent = level.label;
  els.aqiLabel.style.color = level.color;
  els.aqiLabel.style.borderColor = level.color;
  els.cardAQI.style.setProperty("--accent", level.color);
  els.humidityValue.textContent = Math.round(humidity) + "%";
}

function renderWind(speed, deg) {
  els.windSpeed.textContent = Math.round(speed);
  els.windDir.textContent = windDirLabel(deg) + " (" + Math.round(deg) + "°)";
  els.compassNeedle.style.transform = "rotate(" + deg + "deg)";
}

function setLocationLabel(name) {
  els.locationName.textContent = name;
}

function setUpdatedNow() {
  var d = new Date();
  var h = d.getHours() % 12 || 12;
  var m = d.getMinutes();
  var suffix = d.getHours() >= 12 ? "PM" : "AM";
  els.updatedAt.textContent = "Updated " + h + ":" + (m < 10 ? "0" + m : m) + suffix;
}

// ---------------- data loading ----------------

function loadWeather(lat, lon, label) {
  clearError();
  if (label) setLocationLabel(label);

  var forecastUrl = "https://api.open-meteo.com/v1/forecast" +
```

```
      "?latitude=" + lat + "&longitude=" + lon +
      "&current=temperature_2m,apparent_temperature,relative_humidity_2m,weather_code,wind_speed_10m,wind_direction_10
m" +
      "&hourly=precipitation_probability" +
      "&forecast_days=1&temperature_unit=fahrenheit&wind_speed_unit=mph&timezone=auto";

    var aqiUrl = "https://air-quality-api.open-meteo.com/v1/air-quality" +
      "?latitude=" + lat + "&longitude=" + lon +
      "&current=us_aqi&timezone=auto";

    Promise.all([
      fetch(forecastUrl).then(function (r) { if (!r.ok) throw new Error("forecast"); return r.json(); }),
      fetch(aqiUrl).then(function (r) { if (!r.ok) throw new Error("aqi"); return r.json(); }).catch(function () { ret
urn null; })
    ]).then(function (results) {
      var forecast = results[0];
      var aqi = results[1];
      var current = forecast.current;

      renderTemp(current);
      renderRain(forecast.hourly, current.time);
      renderWind(current.wind_speed_10m, current.wind_direction_10m);

      if (aqi && aqi.current && typeof aqi.current.us_aqi === "number") {
        renderAQI(aqi.current.us_aqi, current.relative_humidity_2m);
      } else {
        els.aqiValue.textContent = "—";
        els.aqiLabel.textContent = "N/A";
        els.humidityValue.textContent = Math.round(current.relative_humidity_2m) + "%";
      }

      setUpdatedNow();
    }).catch(function (err) {
      console.error(err);
      showError("Couldn't load weather data. Check your connection and try again.");
    });

    if (!label) {
      reverseGeocode(lat, lon);
    }
  }

  function reverseGeocode(lat, lon) {
    var url = "https://api.bigdatacloud.net/data/reverse-geocode-client?latitude=" + lat + "&longitude=" + lon + "&loc
alityLanguage=en";
    fetch(url).then(function (r) { if (!r.ok) throw new Error("reverse"); return r.json(); })
      .then(function (data) {
        var name = data.city || data.locality || data.principalSubdivision || null;
        if (name) {
          var region = data.principalSubdivisionCode ? data.principalSubdivisionCode.split("-").pop() : (data.countryN
ame || "");
          setLocationLabel(region ? name + ", " + region : name);
        } else {
          setLocationLabel(lat.toFixed(2) + ", " + lon.toFixed(2));
        }
      })
      .catch(function () {
        setLocationLabel(lat.toFixed(2) + ", " + lon.toFixed(2));
      });
  }

  function useDeviceLocation() {
    setLocationLabel("Locating…");
    clearError();
    if (!("geolocation" in navigator)) {
      showError("Geolocation isn't available on this device — search for a city instead.");
      setLocationLabel("Location unavailable");
      openSearch();
      return;
    }
    navigator.geolocation.getCurrentPosition(
      function (pos) {
        loadWeather(pos.coords.latitude, pos.coords.longitude, null);
      },
      function () {
        showError("Location permission denied — search for a city instead.");
        setLocationLabel("Location unavailable");
        openSearch();
      },
      { timeout: 10000, maximumAge: 300000 }
    );
  }

  // ---------------- search ----------------

  var searchTimer = null;

  function openSearch() {
    els.searchPanel.classList.remove("hidden");
    els.searchBtn.classList.add("active");
```

```
    els.searchInput.focus();
  }
  function closeSearch() {
    els.searchPanel.classList.add("hidden");
    els.searchBtn.classList.remove("active");
    els.searchResults.innerHTML = "";
    els.searchStatus.textContent = "";
    els.searchInput.value = "";
  }

  els.searchBtn.addEventListener("click", function () {
    if (els.searchPanel.classList.contains("hidden")) openSearch();
    else closeSearch();
  });

  els.locateBtn.addEventListener("click", function () {
    closeSearch();
    useDeviceLocation();
  });

  els.searchInput.addEventListener("input", function () {
    var q = els.searchInput.value.trim();
    clearTimeout(searchTimer);
    if (q.length < 2) {
      els.searchResults.innerHTML = "";
      els.searchStatus.textContent = "";
      return;
    }
    els.searchStatus.textContent = "Searching…";
    searchTimer = setTimeout(function () {
      var url = "https://geocoding-api.open-meteo.com/v1/search?name=" + encodeURIComponent(q) + "&count=5&language=en
&format=json";
      fetch(url).then(function (r) { return r.json(); })
        .then(function (data) {
          els.searchResults.innerHTML = "";
          var results = data.results || [];
          if (results.length === 0) {
            els.searchStatus.textContent = "No matches found";
            return;
          }
          els.searchStatus.textContent = "";
          results.forEach(function (place) {
            var li = document.createElement("li");
            var parts = [place.name];
            if (place.admin1) parts.push(place.admin1);
            if (place.country) parts.push(place.country);
            li.textContent = parts.join(", ");
            li.addEventListener("click", function () {
              loadWeather(place.latitude, place.longitude, parts.slice(0, 2).join(", "));
              closeSearch();
            });
            els.searchResults.appendChild(li);
          });
        })
        .catch(function () {
          els.searchStatus.textContent = "Search failed — try again";
        });
    }, 350);
  });

  document.addEventListener("click", function (e) {
    if (els.searchPanel.classList.contains("hidden")) return;
    if (!els.searchPanel.contains(e.target) && e.target !== els.searchBtn && !els.searchBtn.contains(e.target)) {
      closeSearch();
    }
  });

  useDeviceLocation();
})();
</script>
</body>
</html>
```

Appendix B: README.md (full source)

README.md

````
# Weather

A single-page mobile weather prototype. Shows current temperature, whether you need an
umbrella in the next few hours, air quality index + humidity, and wind speed/direction — all on
one screen, no scrolling.

Live data comes from [Open-Meteo](https://open-meteo.com/) (forecast, air quality, and
geocoding) plus [BigDataCloud](https://www.bigdatacloud.com/free-api/free-reverse-geocode-to-city-api)
for reverse-geocoding your device location to a city name. Both are free and require no API key,
so the whole thing is a single static `index.html` with no build step, no framework, and no
server.

## Run locally

Just open `index.html` in a browser, or serve the folder with any static file server, e.g.:

```bash
npx serve .
```

## Deploy to GitHub Pages

1. Push this repo to GitHub.
2. In the repo, go to **Settings → Pages**.
3. Under **Build and deployment**, set **Source** to "Deploy from a branch", branch `main`,
   folder `/ (root)`.
4. Save — the page will be live at `https://<username>.github.io/<repo>/` within a minute or two.
````